T13 Context7 & Sequential MCP

Technical Intermediate

Official documentation and multi-step reasoning within reach of a prompt

Context7: versioned documentation on demand

Context7 solves a common problem: Claude knows popular APIs up to its knowledge cutoff, but not their recent versions or exact official patterns. Context7 queries the official documentation for a library and injects the relevant excerpts directly into the context.

Natural triggers: presence of an import statement, question about a specific API, need for a pattern compliant with official documentation.

```
# This type of prompt triggers Context7
"Implement useEffect with cleanup — React 19"
"Add Auth0 auth with Next.js App Router"
"Configure Prisma for nested transactions"
```

When to prefer Context7 over WebSearch: when version precision matters, when WebSearch results are too generic, or when code must follow official patterns without interpretation.

Sequential Thinking: structured analysis

Sequential Thinking brings a multi-step reasoning engine for complex problems. Rather than answering directly, it decomposes the problem into hypotheses, tests them, and builds an argued response step by step.

Invocation: request the installed Sequential Thinking MCP tool for a problem that benefits from explicit steps. `--think-hard` and `--ultrathink` are not Claude Code CLI flags. The prompt word `ultrathink` adds a reasoning instruction without changing API effort; use `/effort` for supported models. Legacy Haiku 4.5 has no effort parameter.

```
# Invoke Sequential for complex debugging
"Why is this API slow? Use Sequential Thinking to analyze it step by step."

# Architecture design
"Design a multi-level cache system with Sequential Thinking."
```

Do not use for: simple fixes, typos, short explanations, single-file tasks. The reasoning overhead is not justified on trivial cases.

Comparison table

Criterion	Context7	Sequential	Native Claude
Optimal use	Library docs	Complex analysis	Simple tasks
Setup	MCP required	MCP required	None
Token cost	Medium	High	Low
Speed	Fast	Slow (reasoning)	Very fast
Precision	Official docs	Structured logic	General knowledge

Effective combination

The two servers complement each other well for architecture or migration tasks:

1. Context7 provides official patterns for the target library
2. Sequential analyzes how to integrate them into the existing system
3. Native Claude executes the file-by-file modifications

```
# Typical Next.js 14 → 15 migration workflow
1. Context7 → official migration patterns (App Router, caching)
2. Sequential → impact analysis on existing codebase
3. Native Claude → edits to affected files
```

Quick installation

```
# Context7 (requires an API key)
npx -y @upstash/context7-mcp --api-key YOUR_API_KEY

# Sequential Thinking (official Anthropic)
npx -y @modelcontextprotocol/server-sequential-thinking
```

Register the server with `claude mcp add --transport stdio --scope project <name> -- npx -y <package>`, using the selected package and its required arguments. Project scope writes `.mcp.json`; user and local scope live in `~/.claude.json`. Inspect the connection with `/mcp`. See MCP installation.